

Library Management System API - Complete Ubuntu Practical Guide

1. Update Ubuntu

Open Terminal and run:
sudo apt update
sudo apt upgrade -y

2. Install MySQL

```
sudo apt install mysql-server -y
mysql --version
sudo systemctl start mysql
sudo systemctl enable mysql
```

3. Install Node.js and npm

```
sudo apt install nodejs npm -y
node -v
npm -v
```

4. Install Postman

```
sudo snap install postman
```

5. Create Database

```
sudo mysql
CREATE DATABASE library_db;
USE library_db;
```

6. Create Tables

Books, Students, and Issues tables with primary keys and foreign keys.

Books Table SQL

```
CREATE TABLE Books (
  book_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(100),
  author VARCHAR(100),
  category VARCHAR(50),
  available_copies INT
);
```

Students Table SQL

```
CREATE TABLE Students (  
  student_id INT AUTO_INCREMENT PRIMARY KEY,  
  student_name VARCHAR(100),  
  department VARCHAR(50),  
  email VARCHAR(100)  
);
```

Issues Table SQL

```
CREATE TABLE Issues (  
  issue_id INT AUTO_INCREMENT PRIMARY KEY,  
  student_id INT,  
  book_id INT,  
  issue_date DATE,  
  FOREIGN KEY(student_id) REFERENCES Students(student_id),  
  FOREIGN KEY(book_id) REFERENCES Books(book_id)  
);
```

7. Insert Sample Data

Insert 5 books, 5 students, and 10 issue records as practiced.

8. Create Node Project

```
mkdir library-api  
cd library-api  
npm init -y  
npm install express mysql2 cors  
touch server.js
```

9. Server Code

Create Express server, connect MySQL, and implement:
POST /books
GET /books
POST /issues
GET /students/:id/issues

10. Run Server

```
node server.js
```

11. Postman Testing

Test:
POST http://localhost:3000/books
GET http://localhost:3000/books
POST http://localhost:3000/issues
GET http://localhost:3000/students/1/issues

Exam Checklist

1. Show database creation.
2. Show tables.
3. Show inserted records.
4. Run server.
5. Demonstrate all 4 APIs.
6. Show request and response in Postman.

9. Server Code — server.js (Complete API)

Express + MySQL2 REST API with 4 endpoints: POST /books, GET /books, POST /issues, GET /students/:id/issues

```
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');
const app = express();
app.use(express.json());
app.use(cors());

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'library_db'
});

db.connect((err) => {
  if (err) {
    console.log(err);
  } else {
    console.log("MySQL Connected");
  }
});

// POST /books - Add a new book
app.post('/books', (req, res) => {
  const { title, author, category, available_copies } = req.body;
  db.query(
    'INSERT INTO Books(title, author, category, available_copies) VALUES (?, ?, ?, ?)',
    [title, author, category, available_copies],
    (err, result) => {
      if (err) return res.status(500).json(err);
      res.json({ message: 'Book Added Successfully' });
    }
  );
});

// GET /books - Get all books
app.get('/books', (req, res) => {
  db.query('SELECT * FROM Books', (err, result) => {
    if (err) return res.status(500).json(err);
    res.json(result);
  });
});

// POST /issues - Issue a book to a student
app.post('/issues', (req, res) => {
  const { student_id, book_id } = req.body;
  db.query(
    'INSERT INTO Issues(student_id, book_id, issue_date) VALUES (?, ?, CURDATE())',
    [student_id, book_id],
    (err, result) => {
      if (err) return res.status(500).json(err);
      res.json({ message: 'Book Issued Successfully' });
    }
  );
});

// GET /students/:id/issues - Get all issued books for a student
app.get('/students/:id/issues', (req, res) => {
  const id = req.params.id;
  db.query(
    `SELECT
      Students.student_name,
```

```

        Books.title,
        Issues.issue_date
    FROM Issues
    JOIN Students ON Issues.student_id = Students.student_id
    JOIN Books    ON Issues.book_id    = Books.book_id
    WHERE Students.student_id = ?`,
    [id],
    (err, result) => {
        if (err) return res.status(500).json(err);
        res.json(result);
    }
    );
});

app.listen(3000, () => {
    console.log('Server Running On Port 3000');
});

```

Library Management System - Sample Data Inserts

Insert 5 Books

```
INSERT INTO Books(title, author, category, available_copies)
VALUES
('Java Programming', 'James Gosling', 'Programming', 5),
('Python Basics', 'Guido Van Rossum', 'Programming', 4),
('DBMS Concepts', 'Korth', 'Database', 3),
('Data Structures', 'Schaum', 'Computer Science', 2),
('Operating System', 'Galvin', 'Computer Science', 6);
```

Insert 5 Students

```
INSERT INTO Students(student_name, department, email)
VALUES
('Rahul Sharma', 'CSE', 'rahul@gmail.com'),
('Amit Verma', 'IT', 'amit@gmail.com'),
('Priya Patil', 'CSE', 'priya@gmail.com'),
('Sneha Joshi', 'ECE', 'sneha@gmail.com'),
('Rohan Gupta', 'MECH', 'rohan@gmail.com');
```

Insert 10 Issue Records

```
INSERT INTO Issues(student_id, book_id, issue_date)
VALUES
(1,1,'2025-06-01'),
(1,2,'2025-06-02'),
(2,3,'2025-06-03'),
(3,1,'2025-06-04'),
(4,5,'2025-06-05'),
(5,4,'2025-06-06'),
(2,2,'2025-06-07'),
(3,3,'2025-06-08'),
(4,1,'2025-06-09'),
(5,5,'2025-06-10');
```

Verification Queries:

```
SELECT * FROM Books;
SELECT * FROM Students;
SELECT * FROM Issues;
```